

Intelligent Navigation of N -Dimensional Manifolds

A Formal Specification for Manifold-Aware Descent
via Turtle Geometry and Local Principal Component Analysis

Eric G. Suchanek, PhD
suchanek@mac.com

WaveRider Project, Flux Frontiers
<https://github.com/Flux-Frontiers/waverider>

April 1, 2026

Abstract

We present a formal specification for navigating N -dimensional embedding spaces by combining turtle geometry with local principal component analysis. The system generalizes the classical 3D turtle to N dimensions via Givens rotations over an orthonormal frame, then couples this with adaptive, position-dependent PCA to discover the tangent space of the data manifold at each step. Gradients are projected onto this tangent space and weighted by eigenvalue magnitude, yielding a form of empirical natural gradient descent that respects the intrinsic geometry of the embedding rather than its ambient coordinate system. We provide complete mathematical definitions, algorithmic specifications, correctness properties, and complexity analysis.

Contents

1	Introduction	3
2	Mathematical Preliminaries	3
3	TurtleND Specification	4
3.1	State	4
3.2	Operations	4
3.3	3D Backward Compatibility	4
4	ManifoldWalker Specification	5
4.1	State	5
4.2	Core Algorithm: Manifold-Aware Descent	5
4.3	Intrinsic Dimensionality Estimation	6
4.4	Eigenvalue Weighting as Natural Gradient	6
5	Coordinate Transform Geometry	6
6	Probing the Loss Landscape	6
7	Complexity Analysis	7

8	Correctness Properties	7
9	Parameters and Tuning	7
10	Discussion	7
10.1	Why Not Cosine Distance?	7
10.2	Relationship to Existing Methods	8
10.3	From Molecular Geometry to Embedding Spaces	8
11	Empirical Validation: CIFAR-10	8
11.1	Intrinsic Dimensionality of CIFAR-10	8
11.2	Architecture Study	9
11.3	Discussion of Results	9
12	Empirical Validation: CIFAR-100	10
12.1	Intrinsic Dimensionality of CIFAR-100	10
12.2	Architecture Study	10
12.3	The Whitney Crossover	11
13	Conclusion	11

1 Introduction

Large language models and other deep learning systems embed discrete symbolic structures into continuous vector spaces of high dimension N . These embedding spaces are not isotropic: the data concentrates on lower-dimensional manifolds $\mathcal{M} \subset \mathbb{R}^N$ whose intrinsic dimensionality $d \ll N$ varies with position. Standard optimization methods—gradient descent, Adam, and their variants—treat all N dimensions equivalently, wasting capacity on off-manifold noise and ignoring the curvature of the data manifold.

We propose an alternative: *manifold-aware navigation* using a generalized turtle geometry. The turtle carries an orthonormal frame that is continuously re-aligned to the local tangent space of $\mathcal{M}$ via PCA on a neighborhood of data points. This transforms the optimization problem from descent in the ambient $\mathbb{R}^N$ to descent along the manifold, suppressing off-manifold components and weighting on-manifold directions by their data variance.

This document specifies two components:

- (i) **TurtleND**: An N -dimensional turtle with position, orthonormal frame, and Givens rotation primitives.
- (ii) **ManifoldWalker**: A navigation system that wraps TurtleND with local PCA steering, eigenvalue-weighted gradient projection, and adaptive dimensionality estimation.

2 Mathematical Preliminaries

Definition 2.1 (Orthonormal Frame). *An orthonormal frame in $\mathbb{R}^N$ is an ordered set of N vectors $\mathbf{F} = (\mathbf{e}_0, \mathbf{e}_1, \dots, \mathbf{e}_{N-1})$ satisfying:*

$$\langle \mathbf{e}_i, \mathbf{e}_j \rangle = \delta_{ij} \quad \forall i, j \in \{0, \dots, N-1\} \quad (1)$$

where δ_{ij} is the Kronecker delta. Equivalently, the matrix $F \in \mathbb{R}^{N \times N}$ with rows $\mathbf{e}_i$ satisfies $FF^\top = \mathbf{I}_N$.

Definition 2.2 (Givens Rotation). *A Givens rotation $G(i, j, \theta)$ in $\mathbb{R}^N$ acts on the plane spanned by coordinates i and j , rotating by angle θ :*

$$G(i, j, \theta) = \mathbf{I}_N + (\cos \theta - 1)(\mathbf{e}_i \mathbf{e}_i^\top + \mathbf{e}_j \mathbf{e}_j^\top) + \sin \theta (\mathbf{e}_i \mathbf{e}_j^\top - \mathbf{e}_j \mathbf{e}_i^\top) \quad (2)$$

where $\mathbf{e}_i, \mathbf{e}_j$ are standard basis vectors. A Givens rotation preserves orthonormality: if F is orthonormal, so is $G \cdot F$.

Definition 2.3 (Data Manifold). *Given a set of embedding vectors $\mathcal{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_M\} \subset \mathbb{R}^N$, the data manifold $\mathcal{M}$ is the smooth submanifold (or the best smooth approximation thereof) on which the data concentrates. The intrinsic dimensionality $d(\mathbf{p})$ at a point $\mathbf{p}$ is the dimension of the tangent space $T_{\mathbf{p}}\mathcal{M}$.*

Definition 2.4 (Local Tangent Space via PCA). *Given a point $\mathbf{p} \in \mathbb{R}^N$ and its k -nearest neighborhood $\mathcal{N}_k(\mathbf{p}) \subset \mathcal{X}$, the empirical local tangent space is the span of the top- d eigenvectors of the local covariance matrix:*

$$\mathbf{C}(\mathbf{p}) = \frac{1}{k-1} \sum_{\mathbf{x} \in \mathcal{N}_k(\mathbf{p})} (\mathbf{x} - \bar{\mathbf{x}})(\mathbf{x} - \bar{\mathbf{x}})^\top \quad (3)$$

where $\bar{\mathbf{x}}$ is the neighborhood centroid. The eigendecomposition $\mathbf{C} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^\top$ with $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_N \geq 0$ yields the principal directions $\mathbf{v}_1, \dots, \mathbf{v}_N$ as columns of $\mathbf{V}$.

3 TurtleND Specification

3.1 State

The TurtleND maintains:

- Position: $\mathbf{p} \in \mathbb{R}^N$
- Frame: $\mathbf{F} = (\mathbf{e}_0, \mathbf{e}_1, \dots, \mathbf{e}_{N-1})$, an orthonormal frame

By convention:

- $\mathbf{e}_0$: *heading* (direction of movement)
- $\mathbf{e}_1$: *left*
- $\mathbf{e}_2$: *up* (when $N \geq 3$)
- $\mathbf{e}_3, \dots, \mathbf{e}_{N-1}$: higher-dimensional basis vectors

Initial state: $\mathbf{p} = \mathbf{0}$, $\mathbf{F} = \mathbf{I}_N$.

3.2 Operations

Table 1: TurtleND operations and their mathematical definitions.

Operation	Definition	Plane
MOVE(d)	$\mathbf{p} \leftarrow \mathbf{p} + d \cdot \mathbf{e}_0$	—
ROTATE(θ, i, j)	Apply $G(i, j, \theta)$ to $\mathbf{F}$	$(\mathbf{e}_i, \mathbf{e}_j)$
TURN(θ)	ROTATE($\theta, 0, 1$)	heading-left
ROLL(θ)	ROTATE($\theta, 1, 2$)	left-up
PITCH(θ)	ROTATE($-\theta, 0, 2$)	heading-up
YAW(θ)	ROTATE($180^\circ - \theta, 0, 1$)	heading-left
TOLOCAL($\mathbf{g}$)	$\ell_i = \langle \mathbf{e}_i, \mathbf{g} - \mathbf{p} \rangle$	—
TOGLOBAL(ℓ)	$\mathbf{g} = \mathbf{p} + \sum_i \ell_i \mathbf{e}_i$	—

Proposition 3.1 (Frame Invariant). *All TurtleND rotation operations preserve the orthonormality of $\mathbf{F}$. That is, if $\mathbf{F}$ is orthonormal before a rotation, it is orthonormal after (up to floating-point precision).*

Proof. Each rotation modifies exactly two rows of F via a 2×2 rotation matrix $\begin{pmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{pmatrix}$ applied to rows i and j . Since the original rows are orthonormal and a rotation matrix preserves inner products, the result is orthonormal. $\square$

Proposition 3.2 (Coordinate Transform Invertibility). *$\text{TOGLOBAL}(\text{TOLOCAL}(\mathbf{g})) = \mathbf{g}$ for all $\mathbf{g} \in \mathbb{R}^N$.*

Proof. Let $\boldsymbol{\delta} = \mathbf{g} - \mathbf{p}$. Then $\text{TOLOCAL}(\mathbf{g}) = F\boldsymbol{\delta}$ and $\text{TOGLOBAL}(F\boldsymbol{\delta}) = \mathbf{p} + F^\top F\boldsymbol{\delta} = \mathbf{p} + \boldsymbol{\delta} = \mathbf{g}$ since $F^\top F = \mathbf{I}_N$. $\square$

3.3 3D Backward Compatibility

Theorem 3.1 (Equivalence). *TurtleND with $N = 3$ produces identical position and frame vectors as Turtle3D for all sequences of MOVE, TURN, ROLL, PITCH, and YAW operations with identical parameters.*

This is verified empirically by exhaustive testing of individual operations and compound sequences.

4 ManifoldWalker Specification

4.1 State

The ManifoldWalker maintains:

- A TurtleND instance (position and frame)
- An embedding matrix $\mathcal{X} \in \mathbb{R}^{M \times N}$
- An objective function $\mathcal{L} : \mathbb{R}^N \rightarrow \mathbb{R}$ to minimize
- Parameters: neighborhood size k , variance threshold τ , learning rate η
- Diagnostics: eigenvalue spectrum $\boldsymbol{\lambda}$, intrinsic dimensionality d

4.2 Core Algorithm: Manifold-Aware Descent

Algorithm 1 ManifoldWalker.Step

Require: Current position $\mathbf{p}$, embeddings $\mathcal{X}$, objective $\mathcal{L}$, parameters k, τ, η

Ensure: Updated position $\mathbf{p}'$, objective value $\mathcal{L}(\mathbf{p}')$

```

1: Phase 1: Discover local manifold geometry
2:  $\mathcal{N} \leftarrow k$ -nearest neighbors of  $\mathbf{p}$  in  $\mathcal{X}$ 
3:  $\bar{\mathbf{x}} \leftarrow \frac{1}{k} \sum_{\mathbf{x} \in \mathcal{N}} \mathbf{x}$ 
4:  $\mathbf{C} \leftarrow \frac{1}{k-1} \sum_{\mathbf{x} \in \mathcal{N}} (\mathbf{x} - \bar{\mathbf{x}})(\mathbf{x} - \bar{\mathbf{x}})^\top$ 
5:  $(\lambda_1, \mathbf{v}_1), \dots, (\lambda_N, \mathbf{v}_N) \leftarrow$  eigendecomposition of  $\mathbf{C}$   $\triangleright \lambda_1 \geq \dots \geq \lambda_N$ 

6: Phase 2: Estimate intrinsic dimensionality
7:  $d \leftarrow \min \left\{ j : \frac{\sum_{i=1}^j \lambda_i}{\sum_{i=1}^N \lambda_i} \geq \tau \right\}$ 

8: Phase 3: Orient turtle to manifold tangent space
9:  $\mathbf{e}_i \leftarrow \mathbf{v}_i$  for  $i = 0, \dots, N-1$   $\triangleright$  Align frame to principal directions

10: Phase 4: Compute and project gradient
11:  $\mathbf{g} \leftarrow \nabla \mathcal{L}(\mathbf{p})$   $\triangleright$  Analytic or numerical
12:  $\boldsymbol{\ell} \leftarrow F \cdot \mathbf{g}$   $\triangleright$  Project into local frame

13: Phase 5: Suppress off-manifold components and weight
14: for  $i = 0, \dots, N-1$  do
15:   if  $i \geq d$  then  $\triangleright$  Off-manifold: suppress
16:      $\ell_i \leftarrow 0$ 
17:   else  $\triangleright$  Eigenvalue weighting
18:      $\ell_i \leftarrow \ell_i \cdot \frac{\lambda_i}{\lambda_1}$ 
19:   end if
20: end for

21: Phase 6: Step in global coordinates
22:  $\mathbf{s} \leftarrow F^\top \cdot \boldsymbol{\ell}$   $\triangleright$  Map weighted gradient back to global
23:  $\mathbf{p}' \leftarrow \mathbf{p} - \eta \cdot \mathbf{s}$   $\triangleright$  Descend

24: return  $\mathbf{p}', \mathcal{L}(\mathbf{p}')$ 

```

4.3 Intrinsic Dimensionality Estimation

The intrinsic dimensionality $d(\mathbf{p})$ is estimated at each step as the number of eigenvalues needed to capture fraction τ of the total variance:

$$d(\mathbf{p}) = \min \left\{ j \in \{1, \dots, N\} : \frac{\sum_{i=1}^j \lambda_i(\mathbf{p})}{\sum_{i=1}^N \lambda_i(\mathbf{p})} \geq \tau \right\} \quad (4)$$

This is *adaptive*: d may vary from point to point on the manifold. Regions of high curvature or branching may exhibit higher d ; flat regions may have very low d .

4.4 Eigenvalue Weighting as Natural Gradient

The weighting $w_i = \lambda_i/\lambda_1$ applied to each on-manifold gradient component implements a form of natural gradient descent. The local covariance $\mathbf{C}(\mathbf{p})$ serves as an empirical estimate of the metric tensor of the data manifold. Directions of high data variance (large λ_i) receive proportionally larger step sizes, while directions of low variance (small λ_i) are damped.

This is analogous to the Fisher Information Matrix (FIM) in classical natural gradient methods, but estimated nonparametrically from the data geometry rather than from the model’s parameter space.

Remark 4.1 (Relationship to Riemannian Optimization). *The ManifoldWalker implements a discrete approximation to Riemannian gradient descent on the data manifold $\mathcal{M}$. At each point:*

1. *The local PCA approximates the tangent space $T_{\mathbf{p}}\mathcal{M}$*
2. *The eigenvalue-weighted projection approximates the Riemannian gradient $\nabla_{\mathcal{M}}\mathcal{L}$*
3. *The linear step approximates the exponential map $\exp_{\mathbf{p}}(-\eta\nabla_{\mathcal{M}}\mathcal{L})$*

The approximation quality depends on the neighborhood size k and the local curvature of $\mathcal{M}$.

5 Coordinate Transform Geometry

The turtle’s frame provides a moving coordinate system. At any point $\mathbf{p}$ with frame F :

$$\text{ToLOCAL}(\mathbf{g}) = F \cdot (\mathbf{g} - \mathbf{p}) \quad (5)$$

$$\text{ToGLOBAL}(\boldsymbol{\ell}) = \mathbf{p} + F^\top \cdot \boldsymbol{\ell} \quad (6)$$

In the ManifoldWalker context, after orientation:

- ℓ_0 : component along direction of maximum local variance (heading)
- ℓ_1 : component along second principal direction (left)
- ℓ_i for $i < d$: on-manifold components
- ℓ_i for $i \geq d$: off-manifold components (suppressed)

6 Probing the Loss Landscape

The PROBE operation evaluates the objective along a single principal direction without moving the turtle:

$$\text{PROBE}(i, \{s_1, \dots, s_m\}) = \{\mathcal{L}(\mathbf{p} + s_j \cdot \mathbf{e}_i)\}_{j=1}^m \quad (7)$$

This enables:

- Visualization of the loss landscape along individual principal components
- Detection of non-convexity or saddle points in specific directions
- Adaptive step size selection based on local curvature estimates

7 Complexity Analysis

Table 2: Computational complexity per step.

Operation	Complexity	Notes
k -NN search	$O(MN)$	Brute force; $O(N \log M)$ with KD-tree
Covariance matrix	$O(kN^2)$	
Eigendecomposition	$O(N^3)$	Full spectrum; $O(N^2d)$ for top- d only
Gradient (numerical)	$O(2N \cdot C_{\mathcal{L}})$	$C_{\mathcal{L}}$ = cost of one $\mathcal{L}$ evaluation
Projection & step	$O(N^2)$	Matrix-vector products
Total per step	$O(MN + N^3 + NC_{\mathcal{L}})$	Dominated by KNN or eigendecomp

For practical embedding spaces ($N \sim 768$ – 4096 , $M \sim 10^4$ – 10^6), the KNN search dominates. This can be mitigated with approximate nearest neighbor structures (FAISS, Annoy, ScaNN). The eigendecomposition can use truncated SVD when only the top- d components are needed.

8 Correctness Properties

- P1. Frame Orthonormality.** The turtle’s frame remains orthonormal after any sequence of rotations: $FF^{\top} = \mathbf{I}_N$. A periodic Gram-Schmidt re-orthonormalization corrects floating-point drift.
- P2. Coordinate Invertibility.** $\text{ToGlobal} \circ \text{ToLocal} = \text{id}$ (Proposition 3.2).
- P3. 3D Compatibility.** $\text{TurtleND}(3)$ is operationally identical to Turtle3D (Theorem 3.1).
- P4. Descent Property.** For a smooth, locally convex objective $\mathcal{L}$ with Lipschitz gradient, and sufficiently small η , each step reduces the objective: $\mathcal{L}(\mathbf{p}') \leq \mathcal{L}(\mathbf{p}) - c \|\nabla_{\mathcal{M}} \mathcal{L}(\mathbf{p})\|^2$ for some $c > 0$ depending on η and the eigenvalue spectrum.
- P5. Manifold Alignment.** The frame’s first d basis vectors span the same subspace as the top- d eigenvectors of the local covariance, ensuring the step direction lies in (or near) $T_{\mathbf{p}}\mathcal{M}$.

9 Parameters and Tuning

10 Discussion

10.1 Why Not Cosine Distance?

Cosine distance treats the embedding space as isotropic—all angular directions are equally meaningful. This is false for learned embeddings. The variance along different principal directions can differ by orders of magnitude, and those directions *rotate* as one moves through the space. The ManifoldWalker’s local PCA captures this anisotropy directly.

Table 3: ManifoldWalker parameters and guidance.

Parameter	Symbol	Default	Guidance
Neighborhood size	k	50	$\sim 5\text{--}10\times$ expected d
Variance threshold	τ	0.95	Lower $\rightarrow$ more aggressive dimensionality reduction
Learning rate	η	0.01	Scale inversely with gradient magnitude
Gradient epsilon	ε	10^{-5}	For numerical differentiation only

10.2 Relationship to Existing Methods

- **Natural Gradient Descent** [5]: Uses the Fisher information matrix; ManifoldWalker uses the empirical data covariance as a nonparametric analog.
- **Local Tangent Space Alignment** [6]: Discovers manifold structure via local PCA; ManifoldWalker adds navigation and optimization.
- **Riemannian Optimization** [7]: Requires knowing the manifold geometry a priori; ManifoldWalker discovers it empirically at each step.

10.3 From Molecular Geometry to Embedding Spaces

TurtleND is a direct generalization of Turtle3D, the local coordinate system at the heart of PROTEUSPY [1], an open-source Python package for protein structure modeling and disulfide bond analysis (Suchanek, 2024). Turtle3D itself descends from PROTEUS, a structure-based molecular modeling program developed as part of the first author’s doctoral thesis (Suchanek, 1990) [2].

The same frame-based navigation that builds proteins from torsion angles can navigate the latent spaces that represent them. Turtle3D moves through $\mathbb{R}^3$ via MOVE, TURN, ROLL, PITCH, and YAW; TurtleND lifts all five operations to $\mathbb{R}^N$ via Givens rotations (Section 3). The ManifoldWalker then couples this frame to local PCA, closing the loop from molecular geometry to high-dimensional machine learning.

proteusPy is available at <https://github.com/suchanek/proteusPy> and published in the *Journal of Open Source Software* [1]. The WaveRider ML stack, including TurtleND and ManifoldWalker, now lives in its own repository at <https://github.com/Flux-Frontiers/waverider>.

11 Empirical Validation: CIFAR-10

We validate the theoretical predictions of Sections 4 and 10 with a controlled architecture study on CIFAR-10. All experiments use raw pixel inputs ($N = 3072$, $32\times 32\times 3$), 30 training epochs, 3 independent trials, and a Metal GPU (TensorFlow 2.16.2). Reported accuracies are means over trials.

11.1 Intrinsic Dimensionality of CIFAR-10

Applying the variance-threshold estimator of Eq. (4) with $\tau = 0.90$ to the CIFAR-10 training set yields a global intrinsic dimensionality of $d^* = 19$. Table 4 shows sensitivity to τ and per-class variation.

Per-class estimation (at $\tau = 0.90$) reveals that d varies across the manifold, from 15.2 (airplane) to 18.0 (truck), consistent with the adaptive $d(\mathbf{p})$ prediction of Section 4. The compression ratio is $N/d^* = 3072/19 \approx 162$.

Table 4: CIFAR-10 intrinsic dimensionality $d(\tau)$ at several variance thresholds (mean $\pm$ std over samples).

τ	Mean d	Median d
0.95	19.4 ± 0.8	20
0.90	16.3 ± 0.9	16
0.85	14.0 ± 0.9	14
0.80	12.1 ± 0.9	12

11.2 Architecture Study

We compare nine architectures spanning the parameter-accuracy frontier. *ManifoldResNet- d* replaces the standard 32-filter count at every residual block with d^* , producing a d^* -dimensional feature vector aligned with the data manifold before the final softmax classifier. *Intrinsic Dim* is a purely linear $\text{PCA} \rightarrow d^* \rightarrow \text{output}$ model with no hidden nonlinearity.

Table 5: CIFAR-10 architecture comparison ($N = 3072$, $d^* = 19$, 30 epochs, 3 trials). $\text{Acc}/\text{Kparam} = \bar{\alpha} / (\text{params}/1000)$.

Architecture	Params	Acc (%)	Acc/Kparam
Standard MLP ($1024 \rightarrow 512$)	3,676,682	19.3	0.0001
Manifold MLP ($2d^* \rightarrow d^*$)	58,587	32.4	0.0055
ResNet (Adam baseline)	47,978	64.2	0.0134
ManifoldResNet-d^*	17,376	64.9	0.0373
ManifoldResNet- $2d^*$	67,232	63.6	0.0095
PCA(100) $\rightarrow$ 10	11,110	33.9	0.0305
PCA(100) Whitney($2d^*$) $\rightarrow$ 10	4,228	36.0	0.0851
PCA(100) linear $\rightarrow$ 10	1,010	39.8	0.3941
Intrinsic Dim ($\text{PCA} \rightarrow d^* \rightarrow \text{output}$)	580	32.2	0.5561

11.3 Discussion of Results

Manifold alignment matches accuracy at $2.8\times$ compression. ManifoldResNet- d^* matches the ResNet baseline (64.9% vs. 64.2%) while using $2.8\times$ fewer parameters (17,376 vs. 47,978). This directly validates P5 (Manifold Alignment): constraining the representation to $T_{\mathbf{p}}\mathcal{M}$ removes wasted capacity without discarding signal.

Over-dimensioning degrades efficiency. ManifoldResNet- $2d^*$ ($d = 38$) uses $4\times$ more parameters than ManifoldResNet- d^* yet achieves *lower* accuracy (63.6% vs. 64.9%). The 19 extra dimensions are off-manifold noise, confirming the eigenvalue suppression in Algorithm 1 Phase 5.

The ambient MLP fails catastrophically. The standard $1024 \rightarrow 512$ MLP (3.7M parameters) achieves only 19.3%, barely above chance, despite being $212\times$ larger than ManifoldResNet- d^* . This is the quantitative cost of ignoring manifold structure in a $161\times$ -compressed space.

Linear intrinsic-dim model outperforms the ambient MLP. The 580-parameter Intrinsic Dim model ($\text{PCA} \rightarrow d^* \rightarrow \text{output}$, no hidden layer) achieves 32.2%—surpassing the 3.7M-parameter MLP by 12.9 percentage points. This is the starkest illustration of Section 1’s claim: working in the intrinsic subspace dominates operating in $\mathbb{R}^N$.

Parameter efficiency frontier. The acc/Kparam column reveals a Pareto frontier. The extreme-parameter models (Intrinsic Dim: 0.556, PCA linear: 0.394) achieve highest efficiency but plateau in absolute accuracy. ManifoldResNet- d^* (0.037) sits at the high-accuracy knee: best absolute performance with the best efficiency among competitive models.

12 Empirical Validation: CIFAR-100

We extend the validation to CIFAR-100 ($N = 3072$, 100 classes, 50 epochs, 3 trials, CPU). The intrinsic dimensionality discovery is identical in method to Section 11; the architecture suite adds ManifoldResNet-d+C (a class-capacity hidden layer after GAP) and $\text{PCA} \rightarrow d^* \rightarrow C \rightarrow C$.

12.1 Intrinsic Dimensionality of CIFAR-100

Table 6: CIFAR-100 intrinsic dimensionality $d(\tau)$ (mean $\pm$ std).

τ	Mean d	Median d
0.95	18.9 ± 1.0	19
0.90	15.7 ± 1.2	16
0.85	13.3 ± 1.2	14
0.80	11.5 ± 1.1	12

The global $d^* = 19$ at $\tau = 0.95$ is *identical* to CIFAR-10 — the same raw pixel manifold, measured the same way. Per-class variation spans 11.2 (sea, a featureless texture) to 18.0 (motorcycle, a structurally complex object), demonstrating that $d(\mathbf{p})$ tracks visual complexity, not category count. The noise fraction is 99.4%–99.6% across all thresholds.

12.2 Architecture Study

Table 7: CIFAR-100 architecture comparison ($N = 3072$, $d^* = 19$, $C = 100$, 50 epochs, 3 trials).

Architecture	Params	Acc (%)	Acc/Kparam
Standard MLP ($1024 \rightarrow 512$)	3,722,852	5.2	0.000014
ManifoldResNet- d^*	19,176	31.9	0.0166
ManifoldResNet- d^* +C	29,276	30.5	0.0104
ResNet (Adam baseline)	50,948	35.5	0.0070
ManifoldResNet-$2d^*$	70,742	38.5	0.0054
PCA Whitney($2d^*$) $\rightarrow$ 100	7,738	15.0	0.0194
Intrinsic Dim ($\text{PCA} \rightarrow d^* \rightarrow \text{output}$)	2,380	12.1	0.0509

12.3 The Whitney Crossover

The CIFAR-100 results reveal a *crossover* relative to CIFAR-10 that is predicted by the Whitney embedding theorem [4].

On CIFAR-10 (10 classes), ManifoldResNet- d^* *beats* the ResNet baseline (64.9% vs. 64.2%) — the intrinsic d^* filters suffice for 10-class boundary geometry.

On CIFAR-100 (100 classes), ManifoldResNet- $2d^*$ *beats* the ResNet baseline (38.5% vs. 35.5%) while ManifoldResNet- d^* trails (31.9%). The Whitney embedding theorem states that a smooth d^* -dimensional manifold can always be smoothly embedded in $\mathbb{R}^{2d^*}$. When the number of class sub-manifolds is small, d^* feature maps suffice to separate them. When the number grows, the inter-manifold boundary geometry requires the full $2d^*$ ambient space to remain non-self-intersecting.

Table 8: The Whitney Crossover: which filter count wins depends on the number of classes C relative to d^* .

Dataset	C	d^*	Winner	Why
CIFAR-10	10	19	ManifoldResNet- d^*	$C \ll d^*$: d^* separates classes
CIFAR-100	100	19	ManifoldResNet- $2d^*$	$C \gg d^*$: Whitney bound binds

This crossover is not a failure of the method — it is the method *correctly diagnosing its own operating regime*. The selection rule is:

$$\text{filter count} = \begin{cases} d^* & \text{if } C \leq d^* \\ 2d^* & \text{if } C > d^* \end{cases} \quad (8)$$

Both choices are principled, data-derived, and parameter-efficient relative to the arbitrary 32-filter conventional ResNet.

The Intrinsic Dim floor holds. The 2,380-parameter Intrinsic Dim model achieves 12.1% on 100 classes — twelve times random chance — with no convolutional structure. The Standard MLP (1,564× more parameters) achieves 5.2%, less than half as well. The manifold hypothesis is not weakened by 100 classes.

13 Conclusion

TurtleND and ManifoldWalker form the geometric core of the **WaveRider** ML stack [3], providing a principled framework for navigating high-dimensional embedding spaces. By continuously re-aligning a carried orthonormal frame to the local data manifold via PCA, the system achieves manifold-aware descent that:

1. Adapts to locally varying intrinsic dimensionality
2. Suppresses off-manifold gradient noise
3. Weights on-manifold directions by data variance (natural gradient)
4. Provides interpretable diagnostics (eigenvalue spectra, intrinsic dim)
5. Enables directional probing of the loss landscape

Empirically (Sections 11–12), architectures derived from this specification reveal a clean and predictable structure across two benchmarks:

- On CIFAR-10 (10 classes), ManifoldResNet- d^* matches a standard ResNet at 2.8× fewer parameters; a 580-parameter linear model in the intrinsic subspace beats a 3.7M-parameter ambient MLP.

- On CIFAR-100 (100 classes), ManifoldResNet- $2d^*$ outperforms the ResNet baseline, with the crossover predicted exactly by the Whitney embedding theorem: use d^* when $C \leq d^*$, use $2d^*$ when $C > d^*$.

These results confirm the core thesis: the assumption of isotropic $\mathbb{R}^N$ wastes capacity, and replacing it with empirical manifold geometry is both theoretically principled and practically effective. The method does not merely perform well — it *knows when to use which geometry*, and its failures are as interpretable as its successes.

References

- [1] Suchanek, E. G. (2024). proteusPy: A Python package for protein structure and disulfide bond modeling and analysis. *Journal of Open Source Software*, 9(100), 6169. <https://doi.org/10.21105/joss.06169>
- [2] Suchanek, E. G. (1990). Computer-aided strategies for protein design. *Biochemistry*, 29(36). <https://doi.org/10.1021/bi00368a023>
- [3] Suchanek, E. G. (2026). WaveRider: Manifold-aware ML stack with TurtleND, Manifold-Walker, and ManifoldResNet. *GitHub repository*, Flux Frontiers. <https://github.com/Flux-Frontiers/waverider>
- [4] Whitney, H. (1944). The self-intersections of a smooth n -manifold in $2n$ -space. *Annals of Mathematics*, 45(2), 220–246.
- [5] Amari, S. (1998). Natural gradient works efficiently in learning. *Neural Computation*, 10(2), 251–276.
- [6] Zhang, Z., & Zha, H. (2004). Principal manifolds and nonlinear dimensionality reduction via tangent space alignment. *SIAM Journal on Scientific Computing*, 26(1), 313–338.
- [7] Absil, P.-A., Mahony, R., & Sepulchre, R. (2008). *Optimization Algorithms on Matrix Manifolds*. Princeton University Press.